Curso de Python

Clase 4 · Estructuras de datos

list · tuple · dict · set
Cuándo usar cada una

Objetivo: que el estudiante pueda elegir la estructura correcta según el problema.

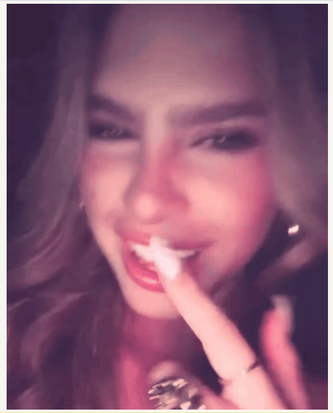
list | tuple | dict | set

El curso hoy responde una sola pregunta:

¿Qué quiero guardar y qué quiero hacer con eso?

Mapa mental

Secuencia · dato fijo · etiquetas · elementos únicos



Ruta de la clase

Mapa

Una presentación segmentada por bloques para dictar la sesión de 1 hora.

7 bloques, una sola meta: decidir bien.

1. Apertura
Por qué elegir bien importa

2. Panorama
Mapa rápido de list / tuple / dict / set

3. Listas
Secuencias que pueden cambiar

4. Tuplas
Secuencias fijas

5. Diccionarios
Datos etiquetados

6. Sets
Elementos únicos y pertenencia

7. Comparación
Casos reales: cuál usar

Bloque 1 · Apertura y motivación

B1

No todas las formas de guardar datos resuelven el mismo problema.

Cuatro situaciones cotidiana

Notas del curso

Guardar varias calificaciones en secuencia.

Coordenada

Representar un dato fijo: (x, y).

Perfil estudiante

Nombre, edad y carrera con significado propio.

Códigos únicos

Quedarse solo con valores sin repetir.

Definición breve: una estructura de datos es una forma de organizar información dentro del programa.

Bloque 1 · La pregunta guía

B1

El hilo conductor que vuelve comparable a list, tuple, dict y set.

¿Importa el orden?

Si la posición tiene sentido, probablemente estoy ante una secuencia.

¿Se puede modificar?

Algunas estructuras cambian; otras comunican estabilidad.

¿Qué quiero guardar y qué quiero hacer con eso?

¿Necesito etiquetas?

Cuando cada dato tiene nombre, conviene trabajar con claves.

¿Quiero evitar repetido

Si la unicidad importa, necesito otra herramienta.

Bloque 2 · Panorama general

B2

Primero un mapa mental; después, el detalle de cada estructura.

Cuatro herramientas, cuatro lógicas de uso

list

[ ]

Colección ordenada que puede cambiar

Pienso en secuencia y edición.

tuple

( )

Colección ordenada que no debería cambiar

Pienso en secuencia y estabilidad.

dict

{ }

Datos en pares clave → valor

Pienso en nombres y valores.

set

{ }

Elementos únicos sin repetidos

Pienso en pertenencia y unicidad.

Clave didáctica: no memorizar primero métodos; entender primero la intención.

Bloque 2 · Tabla comparativa

B2

Las cuatro preguntas clave resumidas en una sola vista.

Comparación rápida

Estructura	Orden	¿Cambia?	Repetidos	Acceso principal
list	Sí	Sí	Sí	Índice
tuple	Sí	No	Sí	Índice
dict	Sí*	Sí	Claves únicas	Clave
set	No	Sí	No	Pertenencia

* En clase conviene pensar dict como estructura “por clave”, aunque conserve el orden de inserción.

Lectura pedagógica

Primero identifica la lógica; después, la sintaxis.

Atajo útil

Secuencia → list / tuple

Atajo útil

Etiqueta → dict · unicidad → set

Bloque 3 · Listas

B3

Regla del bloque: usa list cuando tengas una secuencia ordenada que puede cambiar.

Qué es una lista

- Colección ordenada de elementos.
- Se accede por índice.
- Se puede modificar, agregar y eliminar.
- Permite valores repetidos.

Sintaxis y operaciones básicas

```
notas = [4.5, 3.8, 4.2]

print(notas[0])
notas[1] = 4.0
notas.append(3.5)
notas.remove(4.5)
print(notas)
```

Bloque 3 · Listas en acción

B3

El estudiante debe asociar lista con datos dinámicos.

append · remove · pop · sort

```
tareas = ["estudiar", "almorzar"]

tareas.append("entrenar")
tareas.remove("almorzar")
ultimo = tareas.pop()

numeros = [5, 2, 8, 1]
numeros.sort()
```

Cuándo usarla

- La secuencia puede crecer o reducirse.
- El orden sí importa.
- Necesitas editar elementos durante la ejecución.
- No necesitas etiquetar cada valor con una clave.

Bloque 4 · Tuplas

B4

Regla del bloque: usa tuple cuando el dato no debería cambiar.

Qué es una tupla

- Colección ordenada de elementos.
- También se accede por índice.
- Es inmutable: no se cambia después de crearla.
- Comunica estabilidad del dato.

Sintaxis y acceso

```
coordenada = (3, 5)
fecha = (14, 3, 2026)

print(coordenada[0])
print(coordenada[1])

# coordenada[0] = 10  -> error
```

Bloque 4 · Lista vs tupla

B4

Se parecen en forma; cambian en intención.

list

- Pensada para cambiar.
- Ideal para colecciones dinámicas.
- append, remove, pop, sort.
- Ejemplo: notas o tareas.

tuple

- Pensada para mantenerse fija.
- Ideal para datos estables.
- No se modifica después de crearla.
- Ejemplo: coordenada o fecha.

Pregunta de decisión

¿Este conjunto de datos debería cambiar durante el programa?

Bloque 5 · Diccionarios

B5

Regla del bloque: usa dict cuando quieras asociar una etiqueta con un valor.

Cambio de mentalidad

Ya no pienso en “posición 0”.
Ahora pienso en “nombre”, “edad”, “precio” o “cantidad”.

Clave → valor

```
estudiante = {
    "nombre": "Ana",
    "edad": 20,
    "carrera": "Ingeniería"
}

print(estudiante["nombre"])
estudiante["edad"] = 21
estudiante["ciudad"] = "Bogotá"
```

Bloque 5 · Métodos y usos de dict

B5

Métodos mínimos para empezar sin saturar la clase.

keys · values · items · get

```
estudiante = {"nombre": "Ana", "edad": 20}

print(estudiante.keys())
print(estudiante.values())
print(estudiante.items())
print(estudiante.get("telefono"))
print(estudiante.get("telefono", "No
registrado"))
```

Cuándo un dict mejora la claridad

- Cada valor tiene un significado distinto.
- Quiero consultar por nombre del dato.
- La lectura “persona["edad"]” es más clara que “persona[1]”.
- Necesito representar objetos, registros o configuraciones.

Bloque 6 · Sets

B6

Regla del bloque: usa set cuando importan los elementos únicos y la pertenencia.

Idea central

No pienso en índices.
Pienso en unicidad y en “¿está o no está?”.

Unicidad y pertenencia

```
codigos = [101, 102, 101, 103, 102]

unicos = set(codigos)
print(unicos)

materias = {"física", "programación"}
print("física" in materias)
```

Bloque 6 · Operaciones de conjuntos

B6

El set muestra su valor cuando compara grupos.

Cómo leerlas

Unión

Todo lo que está en A o en B.

Intersección

Lo que ambos conjuntos tienen en común.

Diferencia

Lo que tiene A y no tiene B.

Dif. simétrica

Lo que está en uno u otro, pero no en ambos.

Operaciones básicas

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)
print(a & b)
print(a - b)
print(a ^ b)
```

Aplicación típica

Curso Python = {"Ana", "Luis", "Sara"}
Curso Robótica = {"Luis", "Sara", "Pedro"}

Intersección → {"Luis", "Sara"}

Lectura: el set ayuda a comparar grupos sin depender del orden.

Bloque 7 · ¿Cuál usar en cada caso?

B7

La clave no es la sintaxis; es modelar bien el problema.

Asistencia que puede crecer

list

Coordenada fija

tuple

Datos de un estudiante

dict

Códigos sin repetir

set

Notas del curso

list

Configuración fija

tuple

Inventario de productos

dict

Comparar dos grupos

set

Dinámica sugerida en clase: presentar el caso, dar 10 segundos, escuchar una respuesta y luego justificar.

Bloque 7 · Regla rápida de decisión

B7

La mejor estructura es la que representa mejor la lógica del problema.

Secuencia cambiante

Si el orden importa y la colección se edita, piensa en list.

Secuencia fija

Si el orden importa pero el dato debe permanecer estable, piensa en tuple.

Datos etiquetados

Si cada valor necesita nombre propio, piensa en dict.

Elementos únicos

Si la unicidad o la pertenencia mandan, piensa en set.